

```
tasks = []
categories = []
priorities = []

for i in range(3):
```



작성하신 코드를 보니 투두리스트를 입력받고, 사용자의 선택에 따라 전체/중요도별/카테고리별로 출력하는 흐름을 잘 구현하셨습니다!

하지만 지금 코드는 `for` 문 밖으로 인덴트(들여쓰기)가 잘못 빠져나와 있어서 카테고리 and 중요도는 무조건 1번만 입력받게 되고, `tasks[0]` 처럼 0번 인덱스만 고정해서 조회하고 있는 문제가 있습니다.

`def` 를 사용해 함수로 분리하면 코드가 깔끔해질 뿐만 아니라, 이런 로직 오류도 쉽게 해결할 수 있습니다. 어떤 함수들로 나누면 좋을지 가이드와 수정된 코드를 소개해 드릴게요.

1. 어떤 함수(def)를 만들면 좋을까?

역할(기능)별로 코드를 쪼개는 것이 가장 좋습니다. 크게 입력 / 전체 출력 / 중요도 출력 / 카테고리 출력으로 나눌 수 있습니다.

역할(기능)별로 코드를 쪼개는 것이 가장 좋습니다. 크게 입력 / 전체 출력 / 중요도 출력 / 카테고리 출력으로 나눌 수 있습니다.

- `input_todo()` : 투두리스트 데이터를 입력받아 리스트에 저장하는 함수
- `show_all_tasks()` : 전체 목록을 깔끔하게 출력하는 함수 (1번 메뉴)
- `show_tasks_by_priority()` : 중요도별로 필터링해서 출력하는 함수 (2번 메뉴)
- `show_tasks_by_category()` : 카테고리별로 필터링해서 출력하는 함수 (3번 메뉴)

2. 함수를 적용해 깔끔하게 정리한 코드

기존 코드의 버그(데이터가 1개만 저장되거나 0번만 조회되던 현상)를 해결하고, 반복문(`for`)을 사용해 저장된 모든 데이터를 깔끔하게 출력하도록 수정했습니다.

Python



```
# 전역 변수 선언 (데이터 저장 공간)
tasks = []
categories = []
priorities = []
```

```
# [함수 1] 데이터 입력 기능
```

```
def input_todo():
```

```
priorities = []
```

```
# [함수 1] 데이터 입력 기능
```

```
def input_todo():
```

```
    for i in range(3):
```

```
        print(f'\n--- {i+1}번째 투두리스트 입력 ---')
```

```
        t_input = input('할 일 입력 (그만두려면 "그만" 입력): ')
```

```
        if t_input == '그만':
```

```
            break
```

```
    # 카테고리 와 중요도 입력도 for문 안으로 넣어야 각각 저장됩니다!
```

```
    c_input = input('목록(공부/취미/자기개발/기타): ')
```

```
    p_input = int(input('중요도(1-3): '))
```

```
    tasks.append(t_input)
```

```
    categories.append(c_input)
```

```
    priorities.append(p_input)
```

```
    print(f'"{t_input}" 항목이 저장되었습니다!')
```

```
# [함수 2] 전체 목록 조회 (1번 메뉴)
```

```
def show_all_tasks():
```

```
    print(f'\n[전체 목록 조회]')
```

```
    if not tasks:
```

```
        print("저장된 할 일이 없습니다.")
```

```
        return
```

```
    for i in range(len(tasks)):
```

```
        print(f'할 일: {tasks[i]} | 카테고리: {categories[i]} | 중요도: {priorities[i]}')
```

```
return
```

```
for i in range(len(tasks)):
    print(f'- 할 일: {tasks[i]} | 카테고리: {categories[i]} | 중요도: {priorities[i]}')
```

```
# [함수 3] 중요도 별 목록 조회 (2번 메뉴)
```

```
def show_tasks_by_priority():
    print(f'\n[중요도 별 목록보기]')
    select_p = int(input('조회할 중요도(1-3): '))
```

```
# 중요도 타이틀 출력
```

```
if select_p == 3:
    print('★ 매우 긴급!!! ★')
```

```
elif select_p == 2:
    print('■ 보통 ■')
```

```
else:
    print('◆ 여유~ ◆')
```

```
found = False
```

```
for i in range(len(priorities)):
    if priorities[i] == select_p:
        print(f'- {tasks[i]} (카테고리: {categories[i]})')
        found = True
```

```
if not found:
    print(f'해당 중요도({select_p})와 일치하는 할 일이 없습니다.')
```

3. 무엇이 달라졌나요? (개선 포인트)

- 1 가독성 향상: 메인 실행 흐름(`if-elif-else`)이 함수 이름으로만 이루어져 있어서, 코드가 소설처럼 읽히고 깔끔해졌습니다.
- 2 버그 수정: `c_input` 과 `p_input` 을 `for` 문 안으로 넣어 할 일마다 각각의 카테고리와 중요도를 가질 수 있게 했습니다.
- 3 전체 탐색 (`for i in range(len(tasks))`): 기존에는 `tasks[0]` 만 검사해서 첫 번째 데이터만 보여줬지만, 이제는 반복문을 통해 사용자가 입력한 모든 데이터를 검사하고 출력합니다.
- 4 예외 처리 (`found` 변수): 일치하는 데이터가 하나도 없을 때만 "할 일이 없습니다"라는 메시지가 뜨도록 보완했습니다.

함수 개념을 활용해 현재 남은 할 일 개수로 운세 점수를 계산하고, 사용자의 기분 상태까지 조합하여 행운의 색과 모험가 칭호를 유동적으로 분석해 주는 파이썬 코드를 짜줘.

기분 점수와 남은 할 일 개수를 연동해 운세 점수를 계산하고, 그에 맞는 멋진 모험가 칭호와 행운의 색을 부여하는 파이썬 코드입니다.

요청하신 내용을 바탕으로 운세를 계산하는 함수와 이를 호출해 결과를 보여주는 함수를 분리하고, 잘못된 기분 입력이 들어왔을 때 `while True` 로 재입력받는 유효성 검사 로직까지 깔끔하게 반영했습니다.

💡 갓생 분석 및 운세 매칭 코드

💡 코드의 핵심 포인트

- 1 독립된 연산 함수 (`calculate_fortune_score`): 할 일 개수 (`task_count`)라는 매개변수를 받아서 내부 연산을 처리한 뒤, 오직 `final_score` 결과만을 깨끗하게 리턴합니다. 출력문과 섞이지 않아 코드가 정갈합니다.
- 2 함수 간의 데이터 토스: `show_fortune` 내부에서 `len(tasks)` 를 구한 뒤, 이 값을 연산 함수로 던져

함수 개념을 활용해 현재 남은 할 일 개수로 운세 점수를 계산하고, 사용자의 기분 상태까지 조합하여 행운의 색과 모험가 칭호를 유동적으로 분석해 주는 파이썬 코드를 짜줘.

기분 점수와 남은 할 일 개수를 연동해 운세 점수를 계산하고, 그에 맞는 멋진 모험가 칭호와 행운의 색을 부여하는 파이썬 코드입니다.

요청하신 내용을 바탕으로 운세를 계산하는 함수와 이를 호출해 결과를 보여주는 함수를 분리하고, 잘못된 기분 입력이 들어왔을 때 `while True` 로 재입력받는 유효성 검사 로직까지 깔끔하게 반영했습니다.



갓생 분석 및 운세 매칭 코드

Python



코드 다운로드

```
# 테스트를 위한 가상의 할 일 목록 (현재 남은 할 일: 3개)
tasks = ["파이썬 공부하기", "운동 가기", "방 청소하기"]
```

```
def calculate_fortune_score(task_count):
```

```

def calculate_fortune_score(task_count):
    """현재 남은 할 일 개수를 매개변수로 받아 운세 점수를 연산하는 함수"""
    base_score = 10
    # 할 일이 많을수록 압박감이 크다는 전제로 개당 2점씩 감점
    penalty = task_count * 2
    final_score = base_score - penalty

    # 점수가 0점 밑으로 내려가지 않도록 처리 후 반환
    return max(0, final_score)

def show_fortune():
    """오늘의 기분을 입력받고 행운의 색과 칭호를 분석해주는 함수"""
    print("\n--- [ 🧐 갓생 분석 메뉴 ] 기분 & 행운의 색 ---")

    # [유효성 검사] 지정된 기분 외 다른 값을 입력하면 while True로 무한 재입력
    while True:
        mood_color = input("오늘 아침 눈떴을 때의 기분은 어떤가요? (좋음/보통/나쁨): ")
        if mood_color in ["좋음", "보통", "나쁨"]:
            break
        print("❌ [오류] '좋음', '보통', '나쁨' 중 하나만 정확히 입력해 주세요.\n")

    # 전역 변수인 tasks의 개수를 파악 (복수형 변수 명칭 사전에 유의!)
    current_tasks = len(tasks)

    # 타 함수를 호출하면서 매개변수를 던지고, 반환값(return)을 받아옴
    fortune_score = calculate_fortune_score(current_tasks)

    print(f"\n" + "="*30)

```

```

else:
    # 점수가 낮거나 기분이 '나쁨'일 때 진입
    print("🔴 등급: F등급\n🎭 특별 칭호: [휴식이 필요한 모험가]\n🎮 행운의 색: 🔴 레드")
    print("="*30)

```

```

# 함수 실행
show_fortune()

```

💡 코드의 핵심 포인트

- 1 독립된 연산 함수 (`calculate_fortune_score`): 할 일 개수 (`task_count`)라는 매개변수를 받아서 내부 연산을 처리한 뒤, 오직 `final_score` 결과만을 깨끗하게 리턴합니다. 출력문과 섞이지 않아 코드가 정갈합니다.
- 2 함수 간의 데이터 토스: `show_fortune` 내부에서 `len(tasks)` 를 구한 뒤, 이 값을 연산 함수로 던져 주고 결과값을 `fortune_score` 변수로 돌려받는 유기적인 흐름을 가집니다.
- 3 견고한 예외 처리: 사용자가 실수로 "행복함", "슬픔" 등을 입력하더라도 `while True:` 예약어 규칙을 올바르게 지킨 루프 시스템이 오타를 잡아내고 올바른 입력(좋음/보통/나쁨)을 다시 유도합니다.
- 4 다중 조건 검사 (`and` , `!=`): 조건문 내부에서 활력 점수뿐만 아니라 사용자의 기분 문자열 상태까지 동시에 논리 연산자로 비교하여 칭호를 유동적으로 제어합니다.